

现代编程思想

命令式编程

月兔公开课课程组

循环

- 多次执行同一段代码
- `while`, `for`, `loop`
- 循环结构也是一个表达式

while 循环

```
1. let mut i = 0
2. while i < 3 {
3.     println("hello")
4.     i = i + 1
5. } // 重复输出三次
```

```
fn init {
    let mut i = 0
    while i < 3 {
        println("hello")
        i = i + 1
    }
}
```

```
hello
hello
hello
```

while 循环

```
1. while <条件表达式> {  
2.     <循环体>  
3. }
```

- 当条件为真，执行循环体，并回到条件表达式进入下一次判断
- 当条件为假，退出循环体

while 循环

- 利用变量设定循环条件

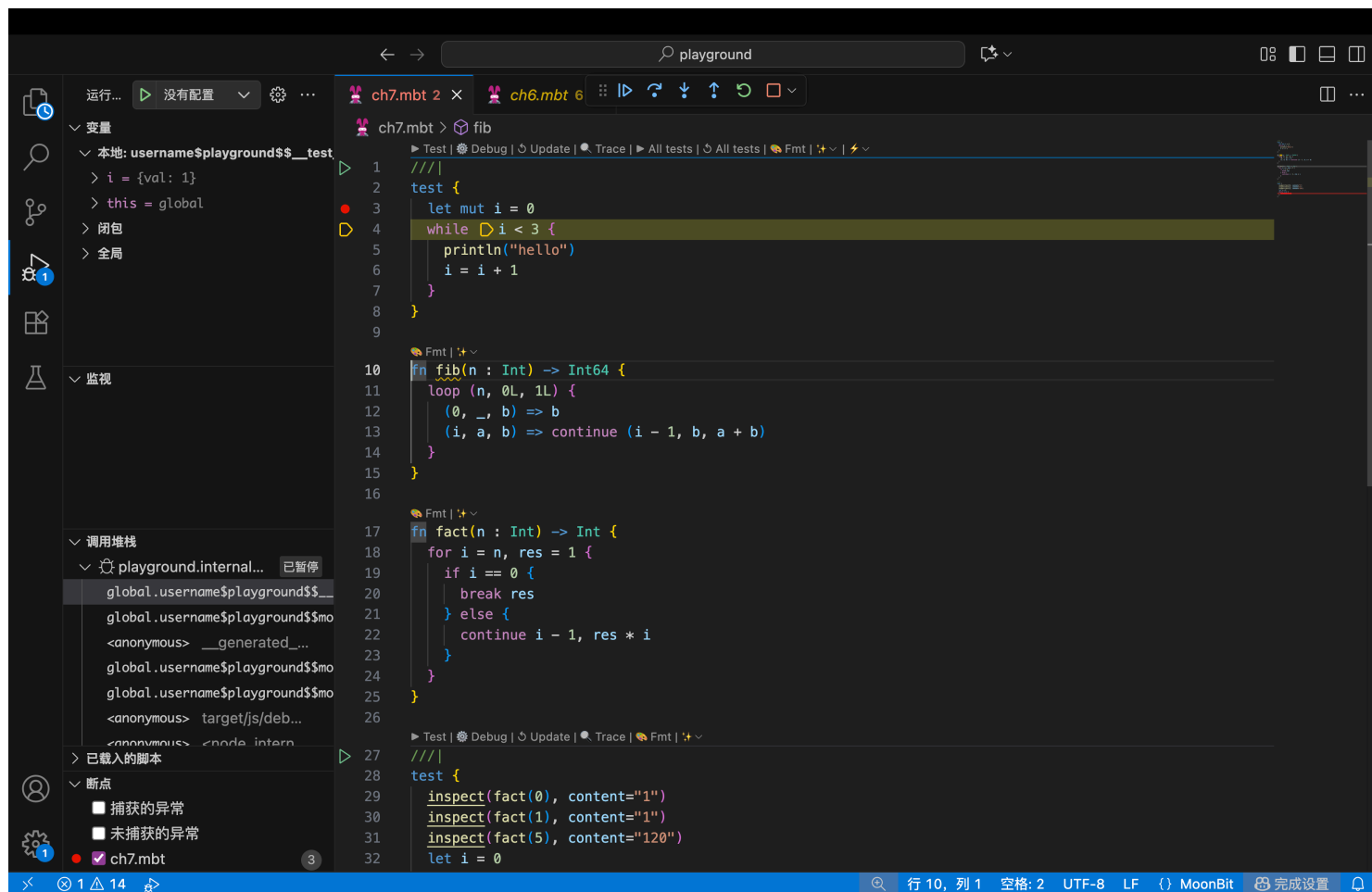
```
1. <定义变量及初始值>
2. while <针对变量判断是否继续循环> {
3.     <需要重复执行的命令>
4.     <对变量进行更新>
5. }
```

- 例如，我们可以反复执行3次输出操作

```
1. let mut i = 0
2. while i < 3 {
3.     println("hello")
4.     i = i + 1
5. } // 重复输出三次
```

调试器

- 月兔的调试器允许我们在运行中看到实时的运行数据，更好理解运行过程



while 循环的 else 块

- 在 while 循环的末尾可以加上一个 else 块
- 循环条件为假时，会执行一次 else 块
- else 块的值即是整个 while 循环表达式的值
- 例子：

```
1. let mut i = 0
2. let n = while i < 3 {
3.     println("hello")
4.     i = i + 1
5. } else {
6.     println("end")
7.     i
8. }
```

循环与递归

- 事实上，循环与递归是等价的

```
1. let mut <变量> = <初始值>
2. while <判断是否继续循环> {
3.     <需要重复执行的命令>
4.     <对变量进行更新>
5. }
```

- 利用可变变量的情况下可以写成

```
1. fn recurse(<参数>) -> Unit {
2.     if <判断是否继续循环> {
3.         <需要重复执行的命令>
4.         recurse(<更新后的参数>)
5.     } else { () }
6. }
7. recurse(<初始值>)
```

循环与递归

- 例如下述两段代码执行效果相同

```
1. let mut i = 0
2. while i < 2 {
3.     println("Hello!")
4.     i = i + 1
5. }
```

```
1. fn recurse(i : Int) -> Unit {
2.     if i < 2 {
3.         println("Hello!")
4.         recurse(i + 1)
5.     } else { () }
6. }
7. recurse(0)
```

循环流的控制

- 循环的时候，可以提前中止循环，或是跳过后续命令的执行
 - `break` 指令可以中止循环
 - `continue` 指令可以跳过后续运行，直接进入下一次循环

```
1. fn print_first_3() -> Unit {  
2.     let mut i = 0  
3.     while i < 10 {  
4.         if i == 3 {  
5.             break // 跳过从3开始的情况  
6.         } else {  
7.             println(i)  
8.         }  
9.         i = i + 1  
10.    }  
11. }
```

循环流的控制

- 循环的时候，可以提前中止循环，或是跳过后续命令的执行
 - `break` 指令可以中止循环
 - `continue` 指令可以跳过后续运行，直接进入下一次循环

```
1. fn print_skip_3() -> Unit {  
2.     let mut i = 0  
3.     while i < 10 {  
4.         if i == 3 {  
5.             continue // 跳过3  
6.         } else { () }  
7.         println(i.to_string())  
8.         i = i + 1  
9.     }  
10. }
```

for 循环

```
1. for <定义变量及初始值>; <判断是否继续循环>; <对变量进行更新> {  
2.     <需要重复执行的命令>  
3. }
```

等价于

```
1. <定义变量及初始值>  
2. while <判断是否继续循环> {  
3.     <需要重复执行的命令>  
4.     <对变量进行更新>  
5. }
```

for 循环

```
1. for i = 0; i < 3; i = i + 1 {  
2.     println("hello")  
3. }
```

- for 循环的变量定义无需用 `let mut`
- for 循环限制了对循环变量的修改

```
1. for i = 0; i < 3; i = i + 1 {  
2.     println("hello")  
3.     i = i + 1 // 不合法! 循环变量不可在此处修改  
4. }
```

for 循环

- 不使用变量更新语句，也可以更新变量。例如

```
1. for i = 0 {  
2.     if i < 3 {  
3.         println("hello")  
4.         continue i + 1  
5.     } else {  
6.         break  
7.     }  
8. }
```

- 这是函数式循环的典型用例

函数式循环

- 函数式 loop 循环和函数式 for 循环
- 不显式为循环变量赋值
- 循环中使用 `break <表达式>` 退出循环，并得到一个值
- 循环中使用 `continue <表达式>` 带着一个值进入下一轮循环

函数式 for 循环

```
1. fn fact(n : Int) -> Int {  
2.   for i = n, res = 1 {           // 初始化循环变量  
3.     if i == 0 {  
4.       break res                 // 将 res 作为整个循环的值  
5.     } else {  
6.       continue i - 1, res * i // 带着更新后的循环变量进入下一轮循环  
7.     }  
8.   }  
9. }  
10.  
11. test {  
12.   inspect(fact(5), content="120")  
13. }
```

loop 循环

- 函数式 for 循环结合模式匹配

```
1. loop <初始值> {  
2.     <模式1> => <表达式1>  
3.     <模式2> => <表达式2>  
4.     ...  
5. }
```

等价于

```
1. for <变量> = <初始值> {  
2.     match <变量> {  
3.         <模式1> => <表达式1>  
4.         <模式2> => <表达式2>  
5.         ...  
6.     }  
7. }
```

loop 循环

```
1. fn fact_with_loop(n : Int) -> Int {  
2.     loop (n, 1) {                                // 初始化循环变量  
3.         (0, res) => break res                      // 将 res 作为整个循环的值  
4.         (i, res) => continue (i - 1, res * i)    // 带着更新后的循环变量进入下一轮循环  
5.     }  
6. }  
7.  
8. test {  
9.     inspect(fact_with_loop(5), content="120")  
10. }
```

月兔的检查

- 月兔会检查一个变量是否被修改，可以避免出现循环忘记更新变量

```

}
Error (warning): The mutability of 'i' is never used, try remove `mut`. (E0015)
Int
fn init { 查看问题 (⌘F8) 快速修复... (⌘.)
  let mut i = 0
  while i < 5 {
    println("hello")
  }
}

```

- 月兔也会检查函数返回结果是否与类型声明相同，可以避免错误的返回类型声明

```

fn plus_one(n : Int) -> Unit {
  n + 1
}
Expr Type Mismatch
  has type : Int
  wanted   : Unit (E4014)
//
te Int
查看问题 (⌘F8) 没有可用的快速修复

```

可变数据

- 使用场景广泛
 - 直接操作程序外环境，如硬件等
 - 一些情况下性能更好，如随机访问数组等
 - 可以构建部分复杂数据结构，如图
 - 重复利用空间（原地修改）
- 可变数据并不总是与引用透明性冲突

```
1. fn fib_mut(n : Int) -> Int { // 对于相同输入，总是有相同输出
2.   let mut acc1 = 0; let mut acc2 = 1; let mut i = 0
3.   while i < n {
4.     let t = acc1 + acc2; acc1 = acc2; acc2 = t
5.     i = i + 1
6.   }
7.   acc1
8. }
```

总结

本章节初步接触了命令式编程，包括

- 如何使用命令
- 如何使用变量
- 如何使用循环